

ZPO – cała wiedza na temat Wzorców w programowaniu obiektowym

Programowanie obiektowe

Programowanie zorientowane obiektowo (ang. Object-Oriented Programming, OOP) to paradygmat programowania, który koncentruje się na modelowaniu rzeczywistości za pomocą obiektów – instancji klas, autonomicznych jednostek zawierających zarówno informacje (atributy), jak i zachowania (metody). W przeciwieństwie do programowania proceduralnego, które opiera się na operacjach wykonywanych na danych, OOP organizuje kod wokół obiektów i ich interakcji.

OOP opiera się na czterech głównych filarach:

- dziedziczenie – mechanizm umożliwiający ponowne wykorzystanie kodu poprzez tworzenie klas pochodnych,
- polimorfizm – możliwość definiowania wielu zachowań dla tych samych interfejsów,
- enkapsulacja – ukrywanie szczegółów implementacji i kontrolowany dostęp do danych,
- abstrakcja – ukrywanie złożoności systemu i eksponowanie tylko istotnych szczegółów.

Klasa

Klasa to podstawowy element OOP, który służy jako szablon do tworzenia obiektów. Definiuje ona strukturę i zachowanie poszczególnych obiektów, określając, jakie atrybuty i metody będą dostępne dla każdej instancji. W Pythonie klasy definiuje się za pomocą słowa kluczowego `class`, a każda instancja klasy jest tworzona poprzez jej wywołanie. Klasy mogą również dziedziczyć po innych klasach, co pozwala na ponowne wykorzystanie kodu i organizowanie go w hierarchiczne struktury.

Dziedziczenie

Dziedziczenie to mechanizm programowania obiektowego, który pozwala jednej klasie (tzw. klasie pochodnej lub podklasie) przejmować właściwości i zachowania innej klasy (tzw. klasy bazowej, nadklasy). Dzięki dziedziczeniu można ponownie wykorzystać istniejący kod, unikać duplikacji i organizować klasy w hierarchie. W Pythonie dziedziczenie realizuje się poprzez podanie nazwy klasy bazowej w nawiasie po nazwie klasy pochodnej. Klasa dziedziczy wszystkie metody i atrybuty klasy nadrzędnej, ale może je także nadpisywać lub rozszerzać.

Polimorfizm

Polimorfizm to jedna z cech OOP pozwalająca różnym klasom definiować metody o tej samej nazwie, ale o różnym działaniu. Takie zachowanie umożliwia pisanie kodu, który działa na obiektach różnych klas w jednolity sposób, co zwiększa jego elastyczność i reużywalność. Polimorfizm sprawia, że metody mogą być nadpisywane lub współdzielone przez różne klasy w spójny sposób.

Enkapsulacja

Enkapsulacja to jedna z cech OOP, która polega na ukrywaniu wewnętrznych szczegółów implementacji klasy oraz kontrolowaniu dostępu do jej atrybutów. Taki zabieg zapewnia większe bezpieczeństwo, modularność i łatwość zarządzania kodem.

Standardowo enkapsulację realizuje się poprzez określenie trzech poziomów dostępu:

- `public` - dostęp publiczny z każdego miejsca w kodzie,

- `protected` - dostęp z poziomu klas pochodnych,
- `private` - dostęp z bieżącej instancji.

Z uwagi na [zacołanie](#) osobiwość języka Python w kontekście enkapsulacji, wszystkie składowe klas są publiczne i nie istnieje żaden sposób na ich ukrycie. Istnieje jednak powszechna konwencja, która wszystkie składowe klas poprzedzone pojedynczym znakiem podkreślenia (np. `_attribute`) traktuje jako prywatne.

Uwaga! Częstym błędem jest "ukrywanie" składowej poprzez podwójne jej podkreślenie (np. `__priv_attr`), co nie czyni jej prywatną, a jedynie zmienia jej referencję w słowniku wartości atrybutów dla danej instancji.

Nowoczesne podejścia do definicji klas

Dataclasses

`dataclasses` to moduł wprowadzony w bibliotece standardowej Pythona w wersji 3.7. Upraszcza on tworzenie klas zorientowanych na przechowywanie informacji. Dzięki zastosowaniu tego modułu nie ma już konieczności manualnego definiowania metod magicznych, takich jak, `__init__`, `__repr__` czy `__eq__`. Dekorator `@dataclass` automatycznie generuje je na podstawie zadeklarowanych atrybutów.

NamedTuple

`namedtuple` to klasa danych z modułu `collections`, która umożliwia tworzenie niezmiennych krotek z nazwanymi atrybutami. Jest używana jako lekka alternatywa dla klas, w sytuacjach wymagających potrzeby przechowywania danych, ale gdy definicja pełnej klasy może być nadmiarowa. `namedtuple` zachowuje wszystkie cechy zwykłej krotki, ale pozwala na dostęp do wartości zarówno przez indeks, jak i przez nazwę pola. `namedtuple`, względem tradycyjnych klas, zużywa mniej pamięci wirtualnej, a także gwarantuje niemodyfikowalność atrybutów zwiększając bezpieczeństwo pamięci.

Pydantic

Pydantic to nowoczesna biblioteka języka Python, która umożliwia walidację i serializację danych na podstawie modeli opartych na `dataclass` i `TypedDict`. Jest szczególnie przydatna w aplikacjach serwerowych, gdzie dane pochodzące z API, baz danych lub plików muszą być bezpieczne i zgodne z oczekiwanym schematem. W odróżnieniu od `dataclass` i `namedtuple`, Pydantic automatycznie sprawdza typy i konwertuje dane wejściowe, co redukuje liczbę błędów związanych z nieprawidłowymi wartościami poszczególnych atrybutów.

Wielodziedziczenie

Wielodziedziczenie (dziedziczenie wielokrotne) to mechanizm w programowaniu obiektowym, który pozwala klasie dziedziczyć po więcej niż jednej klasie bazowej. Dzięki temu można łączyć funkcjonalności z różnych klas, ale jednocześnie wymaga to ostrożności, aby uniknąć konfliktów w metodach i atrybutach. Interpreter języka Python obsługuje dziedziczenie wielokrotne, stosując algorytm MRO (Method Resolution Order), który określa kolejność przeszukiwania klas nadrzędnych w celu znalezienia odpowiedniej metody.

Metody statyczne i klasowe

W języku Python metody w klasach mogą być rodzaju instancyjnego, statycznego lub klasowego. Metody instancyjne (domyślne) operują na konkretnej instancji klasy i mają dostęp do atrybutu `self`. Natomiast metody statyczne i klasowe pozwalają na tworzenie metod, które działają niezależnie od konkretnej instancji, ale w odmienny sposób:

- metoda statyczna - metoda niezwiązana ani z instancją, ani z klasą,
- metoda klasowa - metoda operująca na samej klasie, a nie jej instancjach.

Metody klasowe

Metody klasowe są oznaczone dekoratorem `@classmethod` i działają na poziomie klasy, a nie konkretnej instancji. Jako pierwszy argument przyjmują parametr `cls`, który odnosi się do samej klasy, a nie obiektu. Dzięki temu mogą modyfikować atrybuty klasy i tworzyć jej nowe instancje.

Metody statyczne

Metody statyczne są oznaczone dekoratorem `@staticmethod` działają jak zwykłe funkcje umieszczone wewnątrz klasy – nie mają dostępu do stanu wewnętrznego obiektu (parametr `self`) ani referencji do klasy (parametr `cls`). Służą one do definiowania pomocniczych operacji, które są logicznie związane z klasą, ale nie wymagają dostępu do jej atrybutów.

Klasy abstrakcyjne

W języku Python klasy abstrakcyjne to specjalny rodzaj klas pełniący rolę szablonu dla innych klas. Klas abstrakcyjnych nie można ich bezpośrednio instancjonować. Ich głównym celem jest wymuszenie na klasach dziedziczących implementacji określonych metod oraz spełnienie kontraktu. Taki zabieg umożliwia lepszą organizację kodu.

Moduł `abc` (Abstract Base Classes) będący częścią biblioteki standardowej interpretera języka Python, dostarcza mechanizm do definiowania klas abstrakcyjnych. Używa się go w sytuacji, gdy wszystkie klasy dziedziczące muszą spełniać określony kontrakt.

wzorce kreacyjne

Wprowadzenie do wzorców projektowych

Wzorce projektowe to sprawdzone i powszechnie stosowane rozwiązania typowych problemów projektowych w inżynierii oprogramowania. Opisują one sposób organizacji kodu źródłowego w celu zwiększenia jego czytelności, elastyczności i łatwości utrzymania. Wzorce nie są gotowymi fragmentami kodu, tylko ogólnymi schematami i koncepcjami, które można dostosować do konkretnych potrzeb celem rozwiązania konkretnych problematycznych sytuacji. Wzorce projektowe w najprostszym ujęciu dzielą się na trzy główne kategorie: kreacyjne (dla potrzeb tworzenia obiektów), strukturalne (porządkujące zależności między klasami i obiektami) oraz czynnościowe (definiujące sposoby komunikacji między obiektami).

Krótką historia wzorców projektowych

Idea wzorców projektowych ma swoje korzenie w architekturze budowlanej. W latach 1977 roku wieku architekt i matematyk Christopher Alexander opublikował książkę "A Pattern Language", w której przedstawił koncepcję wzorców jako powtarzalnych i sprawdzonych rozwiązań problemów projektowych w architekturze. Jego podejście inspirowało późniejszych badaczy w obszarze, którzy od

lat '80 XX wieku zaczęli zauważać, że podobnie jak w architekturze, także w programowaniu można zidentyfikować pewne powtarzalne schematy rozwiązywania problemów.

W 1987 roku Ward Cunningham i Kent Beck zaprezentowali ideę wzorców na konferencji OOPSLA, podkreślając ich przydatność w programowaniu zorientowanym obiektowo. Największym kamieniem milowym była jednak książka "Design Patterns: Elements of Reusable Object-Oriented Software" autorstwa Ericha Gamma'ego, Richarda Helma, Ralpha Johnsa i Johna Vlissidesa, która ukazała się w 1994 roku.

W 1996 roku idea wzorców projektowych została rozszerzona do zastosowań w architekturze oprogramowania ukazując się w książce "Pattern-Oriented Software Architecture" autorstwa Franka Buschmann'a, Regine Meunier'a, Hans Rohnert'a, Peter Sommerlad'a oraz Michael Stal'a.

Charakterystyka wzorców kreacyjnych

Wzorce kreacyjne to grupa wzorców projektowych, które odpowiadają za tworzenie nowych obiektów w sposób kontrolowany i elastyczny. Zamiast wykorzystywania standardowych konstruktorów i inicjalizatorów, wzorce te zapewniają bardziej skalowalne i elastyczne rozwiązania, które znacznie ułatwiają utrzymanie kodu, a także perspektywy przyszłej rozbudowy.

Głównym zadaniem wzorców kreacyjnych jest separacja procesu tworzenia kolejnych instancji klas od ich implementacji. Takie podejścia wpływają znacznie na redukcję sztywnych zależności w kodzie źródłowym.

Cechy wzorców kreacyjnych

- dzielenie procesu tworzenia obiektów od ich użycia co pozwala na zmianę sposobu tworzenia obiektów bez wpływu na resztę kodu,
- większa elastyczność i rozszerzalność kodu dzięki dynamicznemu wyborowi odpowiedniej klasy do utworzenia obiektu, dzięki czemu zmiana implementacji obiektu nie wymaga modyfikacji kodu klienta,
- unikanie bezpośredniego użycia operatorów `__new__` oraz `__init__` dzięki czemu tworzenie obiektów odbywa się przez dedykowane metody lub klasy zarządzające,
- ułatwienie zarządzania złożonymi strukturami obiektów dzięki niektórym wzorcom (np. Budowniczy), które pozwalają na stopniowe konstruowanie skomplikowanych obiektów lub umożliwiają klonowanie istniejących obiektów zamiast ich ponownego tworzenia (Prototyp),
- optymalizacja zarządzania pamięcią dzięki wzorcom (np. Singleton), które umożliwiają ponowne użycie obiektów, zmniejszając w ten sposób zużycie pamięci,
- zwiększenie spójności kodu, co ułatwia utrzymanie spójności w sposobie tworzenia obiektów, eliminując duplikowanie fragmentów kodu i niekontrolowane instancjonowanie klas.

Budowniczy

Wzorzec **budowniczy** (ang. builder) służy do tworzenia złożonych obiektów krok po kroku. Oddziela proces tworzenia obiektu od jego reprezentacji, co pozwala na tworzenie różnych wariantów tego samego typu obiektu bez modyfikacji jego struktury.

Przeznaczenie i zastosowanie:

- używany w przypadkach, gdy tworzenie obiektu wymaga wielu kroków lub konfiguracji,
- stosowany w tworzeniu obiektów o wielu opcjonalnych parametrach, eliminując problem z rozbudowanymi konstruktorami lub inicjalizatorami,
- powszechne rozwiązanie w konfiguracji obiektów, tworzeniu dokumentów, generowaniu interfejsów użytkownika i budowaniu złożonych encji w bazach danych.

Wzorzec budowniczy skupia się na sposobie konstrukcji obiektu, a nie na obiektach składowych. Proces tworzenia obiektu jest wręcz często niezależny od mniejszych obiektów, z których się składa. Wzorzec ten używany jest wszędzie tam, gdzie trzeba przejąć kontrolę nad procesem tworzenia obiektu. Proces kończy się, gdy konkretny budowniczy skończy pracę.

Jednym z częstszych realnych zastosowań tego wzorca jest otrzymanie tego samego efektu na wiele sposobów:

- eksport do pliku graficznego w zależności od docelowego formatu,
- szyfrowanie tej samej informacji na wiele różnych sposobów,
- nałożenie filtru na obraz

Wzorzec budowniczy z powodzeniem przejmuje proces tworzenia obiektów, w szczególności gdy ich ostateczna klasa zależy od zewnętrznych czynników. Takie podejście rodzi następujące konsekwencje:

- zwiększona kontrola nad procesem tworzenia obiektów,
- niewielkie ignorowanie obiektów składowych,
- każdy typ produktu wymaga nowego budowniczego,
- brak mamy pewności, że budowniczy zbuduje obiekt ze wszystkich składowych,
- brak pewności, że wszystkie atrybuty budowanego obiektu zostaną zainicjalizowane,
- znaczna objętość kodu.

Metoda wytwórcza

Metoda Wytwórcza (ang. factory method) to wzorzec kreacyjny, który dostarcza interfejs do tworzenia obiektów i pozwala jednocześnie zdecydować podklasom, jaka klasa docelowa zostanie utworzona. Dzięki takiemu podejściu kod bazowy jest niezależny od konkretnych implementacji klas, co sprzyja elastyczności i łatwiejszemu rozszerzaniu systemu. Wzorzec ten znajduje zastosowanie w sytuacjach, gdy dokładny typ tworzonych obiektów może się zmieniać w zależności od kontekstu, a klasa bazowa definiuje ogólną strukturę, delegując szczegóły implementacyjne do klas pochodnych.

Przeznaczenie i zastosowanie:

- oddzielenie logiki tworzenia obiektów od ich zastosowania: wzorzec pozwala na tworzenie obiektów bez konieczności określania ich klas w kodzie głównym,
- łatwe rozszerzanie i modyfikacja kodu: wzorzec zapewnia łatwą możliwość dodawania nowych typów obiektów bez ingerencji w istniejący kod,

- zastosowanie w systemach opartych na interfejsach i polimorfizmie: metoda jest przydatna w implementacjach, które wymagają dynamicznego tworzenia obiektów w zależności od warunków uruchomieniowych.

Metody wytwórcza jest wykorzystywana w przypadkach, gdy istnieje potrzeba oddania podklasom możliwości tworzenia i inicjalizowania obiektów. Takie podejście rodzi pewne konsekwencje:

- podklasy obsługują proces tworzenia obiektów,
- klasa bazowa nie wie dokładnie jaki obiekt został stworzony,
- metoda klasy bazowej może być abstrakcyjna,
- istnieje możliwość łatwego dodania nowej wyspecjalizowanej klasy, która zostanie automatycznie dodana do procesu tworzenia.

Fabryka abstrakcyjna

Fabryka abstrakcyjna (ang. abstract factory) to wzorzec projektowy zapewniający mechanizm tworzenia instancji różnych klas należących do pewnej rodziny bez określania ich konkretnej klasy. Umożliwia on definiowanie interfejsu dla tworzenia obiektów z danej grupy, a właściwe implementacje dostarczają instancje odpowiednich klas. Fabryka abstrakcyjna dzięki temu wspiera zasadę odwrócenia zależności i ułatwia zarządzanie złożonymi systemami poprzez oddzielenie logiki tworzenia obiektów od ich użycia. Fabryka abstrakcyjna jest często stosowana w sytuacjach, gdy istnieje potrzeba dynamicznej zmiany zestawu tworzonych obiektów.

Przeznaczenie i zastosowanie:

- oddzielenie logiki tworzenia obiektów od ich użycia: zmniejsza liczebność zależności w kodzie,
- tworzenie grup powiązanych obiektów: zapewnia spójność w wykorzystaniu zestawów klas,
- modularność kodu: prosta wymiana rodzin obiektów za pomocą zmiany implementacji fabryki.

Założenia początkowe:

- istnieje rodzina fabryk, które zwracają produkt,
- każda fabryka jest realizowana za pomocą innej klasy o tym samym kontrakcie,
- każda fabryka potrafi tworzyć te same rodzaje obiektów,
- każda fabryka zawiera metody o tej samej nazwie,
- celem fabryki abstrakcyjnej będzie dostarczenie odpowiedniej fabryki za pomocą instancji lub referencji.

Fabryka abstrakcyjna jest wykorzystywana w przypadkach, gdy istnieje potrzeba utworzenia instancji różnych klas należących do pewnej rodziny bez określania ich konkretnej klasy. Takie podejście rodzi pewne konsekwencje, które można podzielić na pozytywne i negatywne.

Wady:

- trudności w dodawaniu nowych klas do fabryki,
- wszystkie metody wykonywane są na tej samej klasie/obiekcie.

Zalety:

- hermetyzacja procesu tworzenia obiektów danego typu,
- logika procesu tworzenia obiektów i korzystania z nich jest oddzielona i ogólnodostępna,
- zmniejszona objętość kodu.

Prototyp

Prototyp to wzorzec kreacyjny umożliwiający klonowanie obiektu bez odwoływania się do jego klasy. Dzięki prototypowi można tworzyć inne, takie same, lub podobne obiekty na podstawie pewnego prototypu, który musi posiadać te same atrybuty i metody co obiekty docelowe.

Przeznaczenie i zastosowanie:

- tworzenie nowych obiektów poprzez kopiowanie istniejących instancji,
- zmniejszenie kosztu tworzenia obiektów, szczególnie przydatne, gdy tworzenie nowych instancji jest kosztowne obliczeniowo lub pamięciowo,
- zachowanie struktury i stanu obiektu,
- elastyczność w dynamicznym tworzeniu obiektów: możliwość łatwego tworzenia różnych wariantów obiektów bez konieczności definiowania osobnych klas reprezentujących fabryki.

Wzorzec prototyp jest wykorzystywany wszędzie tam, gdzie istnieje konieczność sklonowania obiektu bez odwoływania się do jego klasy. Takie podejście rodzi pewne konsekwencje:

- możliwość połączenia z fabryką abstrakcyjną,
- wynikiem jest obiekt stworzony na wzór innego obiektu,
- nie należy go stosować gdy istnieje tylko kilka obiektów danego typu (dotyczy to też innych wzorców kreacyjnych),
- każda podklasa prototypu musi implementować metodę clone (właściwie nie dotyczy Pythona),
- w niektórych językach klasy które mogą być klonowane, tylko muszą implementować interfejs/protokół cloneable.

Singleton

Singleton to wzorzec kreacyjny zakładający istnienie co najwyżej N (najczęściej 1) instancji danej klasy, która musi być wszędzie dostępna. Singleton'u należy używać tam, gdzie utworzenie drugiego obiektu tej klasy może mieć negatywne skutki. Zastosowanie wzorca singleton powinno być dokładnie uzasadnione, gdyż jego nadużywanie może stanowić **antywzorzec**.

Przypadki zastosowania:

- logger,
- dziennik zdarzeń,
- bufor,

- obiekt ustawień,
- obiekt łączący z bazą danych lub zewnętrzną usługą,
- szyna danych.

Wzorzec singleton jest wykorzystywany wszędzie tam, gdzie istnieje konieczność kontrolowania liczby instancji klasy. Takie podejście rodzi pewne wady i zalety.

Wady:

- komplikacje przy współbieżności,
- klasa kontroluje instancje oprócz rzeczy które ma na celu,
- trudny w testowaniu.

Zalety:

- szybka implementacja,
- brak zmian po stronie projektu,
- stan singletonów jest globalny,
- leniwe tworzenie instancji: obiekt nie zostanie utworzony, jeżeli nigdy nie zostanie użyty.

Wzorce strukturalne

Charakterystyka wzorców strukturalnych

Wzorce strukturalne (ang. Structural Design Patterns) wspomagają sposób organizacji klas i obiektów w celu optymalnego tworzenia większych i bardziej złożonych struktur o dobrze zdefiniowanych relacjach. Ich głównym celem jest ułatwienie komponowania obiektów, zwiększenie ich modularności oraz zmniejszenie zależności między nimi.

Cechy wzorców strukturalnych:

- łączenie obiektów i klas poprzez definiowanie sposobów, w jakie klasy i obiekty mogą być ze sobą powiązane, aby zapewnić elastyczność podczas projektowania,
- zwiększenie czytelności i rozszerzalności kodu, co wpływa na łatwość jego późniejszego utrzymania,
- zmniejszenie zależności między komponentami poprzez zastosowanie kompozycji (zamiast dziedziczenia), co umożliwia stosowanie zasady luźnego powiązania obiektów,
- wspieranie elastyczności w zarządzaniu obiektami poprzez dynamiczną modyfikację ich działania (dekorator) lub łączenie ich w hierarchiczne struktury (kompozyt),
- optymalizacja wydajności obliczeniowej poprzez sprawne zarządzanie zasobami (pyłek).

Adapter

Wzorzec Adapter służy do konwersji interfejsu jednej klasy na inny interfejs oczekiwany przez drugą klasę (np. klienta). Dzięki takiemu podejściu adapter umożliwia współpracę klas, które inaczej nie mogłyby ze sobą współdziałać z powodu różnych interfejsów. Adapter działa jako pośrednik, tłumacząc wywołania metod z jednej klasy na inny, zgodny format. Wzorzec ten stosuje się głównie w sytuacjach, gdy istnieje potrzeba użycia istniejącego kodu, ale jego interfejs nie pasuje do reszty kodu. Adapter może być zaimplementowany na dwa sposoby: z wykorzystaniem dziedziczenia (Adapter klasowy, który może opakowywać tylko klas) lub kompozycji (Adapter obiektowy, który może opakowywać klasy oraz ich instancje).

Przeznaczenie i zastosowanie

- umożliwienie współpracy klas o niekompatybilnych interfejsach,
- unifikacja interfejsów bibliotek lub modułów,
- hermetyzacja istniejącego kodu bez konieczności jego modyfikacji,
- ułatwienie integracji systemów.

Wzorzec Adapter dokonuje konwersji interfejsów dwóch niekompatybilnych klas zapewniając w ten sposób możliwość interakcji między nimi. Taki proces rodzi pewne konsekwencje:

- współdziałanie dwóch interfejsów klas, które do tej pory nie mogły współpracować,
- zwykle niewielki nakład kodu,
- brak bezpośredniej komunikacji między klasami i obiektami,
- konieczność dostępu klasy adaptera do przynajmniej jednego z obiektów,
- zwiększenie elastyczności implementacji,
- pełna interakcja z zamkniętym kodem klas, w których publiczny jest jedynie interfejs,
- duplikacja części kodu.

Dekorator

Wzorzec Dekorator (ang. Decorator) służy do dynamicznego rozszerzania funkcjonalności obiektów bez modyfikowania ich kodu źródłowego ani wykorzystywania dziedziczenia. Dekorator działa poprzez opakowanie oryginalnego obiektu w inny obiekt będący ostatecznym dekoratorem, który dodaje nowe zachowania przed lub po wywołaniu metod bazowego obiektu. Dekoratory pozwalają na dynamiczne dodawanie, modyfikowanie i łączenie funkcjonalności w sposób, który wspiera zasadę otwarte-zamknięte (OCP - Open/Closed Principle). Są powszechnie stosowane w programowaniu obiektowym oraz w językach wspierających funkcje wyższego rzędu, takich jak Python czy JavaScript.

Przeznaczenie i zastosowanie

- Dynamiczne rozszerzanie funkcjonalności klas obiektów bez konieczności modyfikowania ich kodu.
- Unikanie problemów związanych z zagnieżdżonym i rozgałęzionym dziedziczeniem poprzez możliwość dodawanie nowych zachowań bez tworzenia podklas.

- Zachowanie zgodności z interfejsem obiektu bazowego, co umożliwia łatwe stosowanie dekoratora w istniejącej strukturze kodu źródłowego.

Wzorzec Dekorator dokonuje dynamicznego rozszerzenia funkcjonalności obiektów bez modyfikowania ich kodu źródłowego ani wykorzystywania dziedziczenia. Taki proces rodzi pewne konsekwencje:

- pierwotna implementacja funkcji nie zostaje naruszona,
- brak konieczności dostępu do kodu aby go udekorować,
- python posiada wbudowany mechanizm do dekorowania funkcji,
- dekorowanie odbywa się za każdym razem przy wywołaniu danego obiektu,
- dekorowanie odbywa się dynamicznie po kompilacji kodu,
- użycie dekoratora zajmuje jedną linię kodu,
- jeden z najczęściej używanych wzorców w języku Python.

Fasada

Wzorzec Fasada (ang. Facade) dostarcza uproszczony i jednolity interfejs do bardziej złożonego systemu lub zbioru klas. Jego głównym celem jest ukrycie złożoności wewnętrznej systemu i zapewnienie klientowi prostego sposobu interakcji z nim. Fasada działa jako warstwa pośrednia, która agreguje wywołania do różnych modułów i zarządza ich zależnościami. Takie podejście sprawia, że kod źródłowy staje się bardziej przejrzysty i łatwiejszy w utrzymaniu.

Przeznaczenie i zastosowanie

- Ukrycie skomplikowanej logiki implementacji poprzez dostarczenie uproszczonego interfejsu dla klienta.
- Zmniejszenie zależności między modułami poprzez oddzielenie ich od klienta za pomocą dodatkowej warstwy abstrakcji.
- Poprawa czytelności i utrzymania kodu poprzez organizację skomplikowanych komponentów w warstwy łatwe do zarządzania i utrzymania.
- Ułatwienie migracji lub refaktoryzacji systemów bez wpływu na kod macierzysty.

Wzorzec Fasada dostarcza uproszczony i jednolity interfejs do bardziej złożonego systemu lub zbioru klas. Taki proces rodzi pewne konsekwencje:

- hermetyzacja stosowania skomplikowanego systemu,
- uproszczenie relacji klient-system,
- podział systemu na warstwy tworzące hierarchię fasad,
- powstanie klasy pośredniej pomocniczej,
- zwiększenie bezpieczeństwa systemu poprzez ukrycie macierzystych implementacji.

Kompozyt

Wzorzec Kompozyt (ang. Composite) umożliwia reprezentowanie hierarchicznych struktur obiektów, w których zarówno pojedyncze elementy lub i ich złożone grupy mogą być traktowane w jednolity sposób. Kompozyt definiuje wspólny i spójny interfejs dla obiektów prostych (liści drzewa) oraz obiektów złożonych (kontenerów), co pozwala na ich rekurencyjne przetwarzanie. Dzięki temu klient nie musi rozróżniać, czy pracuje z pojedynczym elementem, czy z całą strukturą – może używać ich w ten sam sposób. Kompozyt jest często stosowany w sytuacjach, gdzie system wymaga modelowania drzewiastej struktury, np. plików w systemie, interfejsów graficznych czy hierarchii organizacyjnych.

Przeznaczenie i zastosowanie

- Ujednolicenie operacji na pojedynczych obiektach i w grupach, umożliwiając traktowanie ich w ten sam sposób.
- Reprezentowanie hierarchii obiektów w postaci drzewa, np. struktury katalogów.
- Zwiększenie elastyczności i skalowalności kodu umożliwiając w ten sposób łatwe rozszerzanie struktury bez zmiany kodu klienta.

Wzorzec Kompozyt umożliwia reprezentowanie hierarchicznych struktur obiektów, w których zarówno pojedyncze elementy lub i ich złożone grupy mogą być traktowane w jednolity sposób. Taki proces rodzi pewne konsekwencje:

- rekurencyjne wykonania metod komponentów,
- reprezentacja obiektów za pomocą struktury drzewa,
- łatwe wykonanie metod zbiorczych, takich jak np. "przesuń wszystkie punkty, które tworzą odcinek i trójkąt",
- ciągła mamy możliwość odwołania się do komponentów składowych,
- dodanie elementów pośrednich bez konieczności przebudowy całego projektu,
- implementacje wielu komponentów mogą być zbyt ogólne: np. klasa Point posiada metody add i remove poprzez dziedziczenie, które nie są wykorzystywane.

Most

Wzorzec Most (ang. Bridge) to wzorzec strukturalny służący do oddzielenia abstrakcji od jej implementacji, umożliwiając ich niezależny rozwój. Dzięki takiemu podejściu, zmiany w jednej części systemu (np. interfejsie użytkownika) nie wpływają na drugą część (np. logikę biznesową). Wzorzec Most pomaga w eliminacji sztywnego powiązania między klasami, co poprawia modularność i elastyczność kodu. Struktura wzorca składa się z dwóch hierarchii: abstrakcji (np. interfejs użytkownika) i implementacji (np. konkretne mechanizmy działania), które komunikują się poprzez interfejs mostu. Takie podejście przydatne jest w przypadkach, gdy istnieje potrzeba dynamicznego wyboru implementacji lub gdy system ma różne warianty działania.

Przeznaczenie i zastosowanie

- Oddzielenie abstrakcji od implementacji komponentów, umożliwiając ich niezależną modyfikację i rozwój.

- Zapewnienie elastyczności kodu poprzez wybór implementacji w trakcie działania programu.
- Redukcja liczby klas w przypadku konieczności obsługi wielu kombinacji abstrakcji i implementacji.
- Ułatwienie testowania i utrzymania kodu poprzez zmniejszenie zależności między klasami.

Wzorzec Most służy do oddzielenia abstrakcji struktur od jich implementacji, umożliwiając ich niezależny rozwój. Taki proces rodzi pewne konsekwencje:

- warty do zastosowania gdy istnieją minimum 2 implementacje pewnego interfejsu,
- jego zastosowanie musi być z góry założone, najlepiej na samym początku implementacji,
- ma wysoką złożoność (połączenie metody szablonowej i strategii),
- brak wielopiętrowych dziedziczeń.

Proxy

Wzorzec Pełnomocnik (ang. Proxy) dostarcza zastępczy obiekt, który kontroluje dostęp do innego obiektu. Pełnomocnik może opóźniać inicjalizację, kontrolować uprawnienia, zapewniać zdalny dostęp lub buforować wyniki w celu optymalizacji działania systemu. Wzorzec Pełnomocnik ten jest powszechnie stosowany w sytuacjach, gdy bezpośredni dostęp do obiektu docelowego jest kosztowny lub wymaga dodatkowej kontroli. Pełnomocnik zachowuje ten sam interfejs co oryginalny obiekt, dzięki czemu klient może używać go bez świadomości, że komunikuje się z pełnomocnikiem zamiast z rzeczywistym obiektem.

Istnieją 4 wersje wzorca pełnomocnik:

- zdalny (remote proxy): istnieje jako lokalna reprezentacja obiektu znajdującego się na zewnątrz, np. na serwerze,
- wirtualny (virtual proxy): imituje finalny obiekt, choć ten zostanie stworzony dopiero w momencie gdy będzie potrzebny,
- ochraniający (protective proxy): kontroluje dostęp do obiektu,
- sprytne odwołanie (smart proxy): dokonuje dodatkowych akcji podczas do obiektu.

Przeznaczenie i zastosowanie

- Kontrola dostępu poprzez umożliwienie lub zablokowanie operacji na obiekcie docelowym.
- Optymalizacja wydajności poprzez opóźnianie inicjalizacji lub buforowanie wyników operacji.
- Obsługa zdalnych zasobów za sprawą umożliwienia dostępu do obiektów znajdujących się na serwerach lub w innych systemach.
- Monitorowanie operacji poprzez rejestrowanie lub modyfikowanie wywołań metod obiektu.
- Bezpieczeństwo poprzez ograniczenie dostępu do zasobów na podstawie autentykacji i autoryzacji użytkownika.

Wzorzec pełnomocnik dostarcza zastępczy obiekt, który kontroluje dostęp do innego obiektu. Taki proces rodzi pewne konsekwencje:

- zwiększone bezpieczeństwo,
- powstanie dodatkowej warstwy dostępowej do obiektu,
- pozostałe konsekwencje zależne od wersji pełnomocnika.

Pyłek

Wzorzec Pyłek (ang. Flyweight) jest przeznaczony do optymalizacji wykorzystania pamięci, poprzez współdzielenie pewnej liczby podobnych obiektów zamiast tworzenia ich osobnych instancji. Stosuje się go w systemach, gdzie istnieje wiele obiektów o podobnych wartościach atrybutów, które można przechowywać wspólnie. Wzorzec ten dzieli atrybuty obiektu na część współdzieloną (stan wewnętrzny), która jest zapisywana raz i używana przez wiele instancji, oraz część unikalną (stan zewnętrzny), która pozostaje specyficzna dla każdej instancji. Wzorzec Pyłek znacząco zmniejsza zużycie pamięci i poprawia wydajność, szczególnie w dużych aplikacjach.

Przeznaczenie i zastosowanie

- Mniejsze zużycie pamięci poprzez współdzielenie wspólnych danych między obiektami.
- Optymalizacja wydajności w przypadkach gdy istnieje wiele podobnych obiektów.
- Separacja wewnętrznego i zewnętrznego stanu, umożliwiając ponowne wykorzystanie danych.
- Poprawa skalowalności aplikacji, szczególnie w aplikacjach przetwarzających duże ilości danych.

Wzorzec Pyłek jest przeznaczony do optymalizacji wykorzystania pamięci, poprzez współdzielenie pewnej liczby podobnych obiektów zamiast tworzenia ich osobnych instancji. Taki proces rodzi konsekwencje:

- oszczędność pamięci poprzez współdzielenie pyłków,
- obiekty muszą być tworzone w specjalnej fabryce,
- odseparowanie części atrybutów pewnego obiektu do różnych klas,
- do zastosowania w systemach konsumujących znaczne ilości pamięci.

wzorce czynnościowe

Charakterystyka wzorców czynnościowych

Wzorce czynnościowe (ang. behavioral patterns) to rodzaj wzorców projektowych, które dotyczą definiowania sposobów współpracy między obiektami oraz podziału obowiązków w ramach systemu. Ułatwiają one komunikację między komponentami, zwiększając w ten sposób elastyczność i reużywalność kodu. Ich główną rolą jest uproszczenie zarządzania przepływem sterowania i interakcjami między warstwami w obrębie aplikacji.

Cechy wzorców czynnościowych:

- podział odpowiedzialności poprzez separację logiki komunikacji od logiki biznesowej,
- elastyczność komunikacji poprzez umożliwienie luźnych powiązań między obiektami,

- unikanie zależności poprzez ich redukcję między komponentami,
- łatwość rozszerzania poprzez umożliwienie dodawania nowych zachowań bez modyfikacji istniejącego kodu,
- obsługa zdarzeń i stanów, które często są stosowane do zarządzania przepływem sterowania w systemach zdarzeniowych.

Strategia

Strategia (ang. Strategy) to wzorzec czynnościowy, który pozwala na definiowanie rodzin algorytmów, enkapsulowanie ich i wybór odpowiedniego z nich w czasie działania programu. Strategia polega na wydzieleniu algorytmów do osobnych klas i wykorzystaniu interfejsu do ich abstrakcyjnej obsługi. Dzięki takiemu podejściu można dynamicznie zmieniać sposób działania obiektu bez ingerowania w jego kod.

Przeznaczenie i zastosowanie

- Oddzielenie logiki wyboru algorytmu od jego implementacji.
- Umożliwienie dynamicznej zmiany metody rozwiązywania danego problemu w czasie działania programu.
- Redukcja złożoności klasy poprzez delegowanie logiki algorytmów do osobnych klas.
- Ułatwienie testowania kodu poprzez możliwość zamiany implementacji strategii.

Przykłady zastosowań

- Kompresja danych i wybór optymalnej metody w trakcie działania procesu.
- Dynamiczny wybór metody sortującej wektor na podstawie jego długości.
- Metoda przedostania się między punktami na ziemi na podstawie drogi, odległości i ukształtowania terenu.

Strategia to wzorzec czynnościowy, który pozwala na definiowanie rodzin algorytmów, enkapsulowanie ich i wybór odpowiedniego z nich w czasie działania programu. Takie podejście rodzi konsekwencje:

- automatyczny lub manualny dobór metody rozwiązania problemu,
- powstanie dodatkowej warstwy wyboru metody,
- eliminacja instrukcji warunkowych po stronie klienta,
- jednakowy wynik,
- powstaje osobna klasy lub funkcji dla każdej ze strategii.

Iterator

Iterator to czynnościowy wzorzec projektowy, który umożliwia sekwencyjny dostęp do elementów kolekcji bez ujawniania jej wewnętrznej struktury. Dzięki takiemu podejściu iterowanie po różnych strukturach danych (np. listach, tablicach, kolejkach) odbywa się w jednolity sposób, niezależnie od ich implementacji. Iterator dostarcza metody do przechodzenia przez kolekcję, często w postaci

interfejsu zgodnego z instrukcjami iteracyjnymi, co zwiększa stopień enkapsulacji i ułatwia implementację wzorca pojedynczej odpowiedzialności.

Przeznaczenie i zastosowanie

- Abstrakcja procesu iterowania po kontenerach danych.
- Oddzielenie logiki iteracji od struktury danych.
- Ułatwienie implementacji różnych metod iteracji (np. w przód, w tył, warunkowo, itd).
- Umożliwienie jednoczesnego przeglądania po kolekcji przez wiele iteratorów.
- Poprawa czytelności kodu poprzez eliminację manualnego zarządzania indeksami.

Iterator to czynnościowy wzorec projektowy, który umożliwia sekwencyjny dostęp do elementów kolekcji bez ujawniania jej wewnętrznej struktury. Takie podejście prowadzi do konsekwencji:

- dostarczanie wyników w czasie rzeczywistym (generator),
- brak przechowywanych wyników w pamięci (generator),
- możliwość zwrócenia dowolnej ilości wyników,
- możliwość wykorzystania instrukcji iteracyjnej do przejścia po elementach.

Metoda szablonowa

Metoda Szablonowa (ang. Template Method) to czynnościowy wzorec projektowy, który definiuje szkielet algorytmu w klasie bazowej, pozostawiając implementację niektórych kroków w klasach pochodnych. Dzięki takiemu podejściu umożliwia ponowne wykorzystanie wspólnej logiki oraz zapewnia spójność procesu, jednocześnie pozwalając na dostosowanie poszczególnych etapów działania. Klasa bazowa zazwyczaj zawiera metodę szablonową, która określa kolejność wywołań metod abstrakcyjnych lub opcjonalnych metod hook, umożliwiając różnym implementacjom zachowanie określonej struktury.

Przeznaczenie i zastosowanie

- Standaryzacja szkieletu algorytmu przy jednoczesnym umożliwieniu dostosowania poszczególnych jego fragmentów.
- Ponowne wykorzystanie wspólnej logiki i redukcja duplikacji kodu.
- Ułatwienie utrzymania i rozszerzania implementacji poprzez rozdzielenie z algorytmu części ogólnych i specyficznych.
- Wymuszenie w algorytmie określonej kolejności wykonywania kroków.

Metoda Szablonowa to czynnościowy wzorec projektowy, który definiuje szkielet algorytmu w klasie bazowej, pozostawiając implementację niektórych kroków w klasach pochodnych. Takie podejście rodzi konsekwencje:

- powstanie nadklasy abstrakcyjnej,
- posiadanie metody uruchamiającej wszystkie kroki danego algorytmu, co stanowi główną istotę wzorca,

- unikanie duplikowania kodu.

Pamiętka

Pamiętka (ang. Memento) to czynnościowy wzorzec projektowy, który umożliwia zapisywanie i przywracanie wcześniejszych stanów obiektu bez naruszania jego hermetyzacji. Wzorzec składa się z trzech głównych elementów: Pamiętki, która przechowuje zapisany stan obiektu, obiektu oryginalnego, który tworzy i odtwarza pamiętki oraz opiekuna, który zarządza pamiętkami i decyduje, kiedy je przywrócić. Dzięki takiemu podejściu możliwe jest implementowanie operacji cofania (ang. undo) i powtarzania (ang. redo), a także zachowanie jednoczesnej enkapsulacji wewnętrznych stanów obiektów.

Przeznaczenie i zastosowanie

- Umożliwienie cofania zmian w obiektach poprzez przywracanie wcześniejszych stanów.
- Zachowanie hermetyzacji obiektu, gdzie jego wewnętrzny stan jest zapisywany i odtwarzany bez ujawniania szczegółów implementacyjnych.
- Odseparowanie logiki zapisu i przywracania stanu od implementacji samego obiektu.
- Umożliwienie śledzenia i odtwarzania historii zmian bez ingerencji w strukturę bazowego obiektu.

Pamiętka to czynnościowy wzorzec projektowy, który umożliwia zapisywanie i przywracanie wcześniejszych stanów obiektu bez naruszania jego hermetyzacji. Takie podejście prowadzi do pewnych konsekwencji:

- eliminacja potrzeby tworzenia wielu instancji tego samego obiektu w celu przechowywania jego stanu,
- zwiększenie złożoności pamięciowej,
- oddelegowanie procesu wczytywania i zapisu stanu do zewnętrznego obiektu.

Łańcuch zobowiązań

Łańcuch Zobowiązań (ang. Chain of Responsibility) to czynnościowy wzorzec projektowy, który umożliwia przekazywanie żądania przez sekwencję potencjalnych obsługujących (handlerów) bez potrzeby jawnego określania, który z nich je obsłuży. Każdy handler decyduje, czy obsłużyć żądanie lub przekazać je dalej w łańcuchu. Dzięki takiemu podejściu możliwe jest dynamiczne kształtowanie procesu obsługi żądań i uniknięcie sztywnego powiązania między nadawcą a odbiorcą.

Zastosowanie i przeznaczenie

- Eliminacja bezpośrednich powiązań między nadawcą, a odbiorcami żądania.
- Umożliwienie dynamicznego dodawania lub usuwania elementów obsługujących żądania.
- Zapewnienie elastyczności w obsłudze żądań poprzez kaskadowe delegowanie odpowiedzialności.

Łańcuch Zobowiązań to czynnościowy wzorzec projektowy, który umożliwia przekazywanie żądania przez sekwencję potencjalnych obsługujących (handlerów) bez potrzeby jawnego określania, który z nich je obsłuży. Takie podejście rodzi konsekwencje:

- ignorowanie lub oddelegowywanie zadań dalej w określonej hierarchii,
- ogniwa łańcucha można modyfikować w sposób dynamiczny,
- zadanie zostanie wykonane przez pierwszy obiekt w hierarchii potrafiący je obsłużyć,
- mogą istnieć zadania nieobsłużone gdy nie istnieje domyślny handler,
- każdy element łańcucha powinien znać co najwyżej 2 sąsiednie elementy.

Polecenie

Polecenie (ang. Command) to czynnościowy wzorzec projektowy, który enkapsuluje żądanie jako obiekt, pozwalając na jego parametryzację, kolejkovanie, logowanie oraz późniejsze wykonanie. Wzorzec polecenie oddziela nadawcę polecenia od jego odbiorcy, co umożliwia dynamiczne zarządzanie działaniami, np. poprzez implementację operacji cofania (undo) lub wykonywania zadań w kolejności. Wzorzec ten składa się z czterech kluczowych elementów: polecenia, które określa akcję do wykonania; odbiorcy, który wykonuje akcję; wywołującego, który inicjuje polecenie oraz klienta, który konfiguruje polecenie.

Przeznaczenie i zastosowanie

- Oddzielenie obiektów wysyłających żądania od obiektów, które je wykonują.
- Możliwość kolejkovania, planowania i logowania poleceń.
- Umożliwienie dynamicznej konfiguracji i komponowania poleceń.

Polecenie to czynnościowy wzorzec projektowy, który enkapsuluje żądanie jako obiekt, pozwalając na jego parametryzację, kolejkovanie, logowanie oraz późniejsze wykonanie. Takie podejście rodzi konsekwencje:

- dana komenda jest uruchamiana na żądanie,
- czynność wykonywana na obiekcie jest w postaci odseparowanej (jako osobny obiekt),
- obiekt wywołujący nie wie w jaki sposób działa ostateczny obiekt,
- polecenia można uruchamiać zarówno sekwencyjnie, jak i równolegle.

Obserwator

Obserwator (ang. Observer) to czynnościowy wzorzec projektowy, który umożliwia automatyczne powiadamianie wielu obiektów (obserwatorów) o zmianach stanu obiektu obserwowanego. Dzięki takiemu podejściu obserwatorzy są informowani o zmianach bez konieczności ciągłego sprawdzania stanu obiektu obserwowanego, co poprawia wydajność i luźne powiązanie między komponentami. Obserwator jest często stosowany w implementacji wzorca publisher-subscriber, gdzie obiekty mogą dynamicznie rejestrować się i wyrejestrowywać jako obserwatorzy.

Przeznaczenie i zastosowanie

- Umożliwienie automatycznego zawiadamiania wielu obserwatorów o zmianach w stanie obiektu obserwowanego.
- Zmniejszenie sztywnego powiązania między obiektem obserwowanym, a jego obserwatorami.

- Usprawnienie wydajności poprzez eliminację potrzeby ciągłego sprawdzania zmian.
- Dynamiczne dodawanie i usuwanie obserwatorów w trakcie działania programu.

Obserwator (ang. Observer) to czynnościowy wzorzec projektowy, który umożliwia automatyczne powiadamianie wielu obiektów (obserwatorów) o zmianach stanu obiektu obserwowanego. Takie podejście powoduje konsekwencje:

- obserwatora i obserwowanego łączy tylko fakt obserwowania i nie muszą mieć podobnej struktury,
- usunięcie obserwatora nie jest możliwe jeśli ten znajduje się na liście u obserwowanego (mogą wystąpić wycieki pamięci),
- zmiany u obu stron nie wpływają na relację obserwowania,
- lista obiektów obserwujących może być zmieniana podczas działania kodu,
- lista obiektów obserwujących może być sortowana np. względem priorytetów zawiadamiania,
- obserwowany wysyła powiadomienie do wszystkich obserwatorów,
- obserwatorzy nie wiedzą o sobie wzajemnie.

Stan

Stan (ang. State) to czynnościowy wzorzec projektowy, który umożliwia zmianę zachowania obiektu w zależności od jego aktualnego stanu. Zamiast stosowania złożonych instrukcji warunkowych, stan deleguje zachowanie do osobnych klas reprezentujących konkretne stany. Dzięki takiemu podejściu obiekt może dynamicznie przełączać się między różnymi stanami bez ingerencji w jego logikę biznesową. Każdy stan implementuje wspólny interfejs.

Przeznaczenie i zastosowanie

- Eliminacja nadmiarowych instrukcji warunkowych związanych z obsługą wielu stanów.
- Umożliwienie dynamicznej zmiany zachowania obiektu w trakcie działania programu.
- Ułatwienie zarządzania złożonymi maszynami stanów poprzez separację logiki dla poszczególnych stanów.
- Poprawa jakości kodu oraz możliwości jego utrzymania poprzez enkapsulację stanów w osobnych klasach.

Stan (ang. State) to czynnościowy wzorzec projektowy, który umożliwia zmianę zachowania obiektu w zależności od jego aktualnego stanu. Takie podejście prowadzi do konsekwencji:

- manualny dobór metody na podstawie stanu obiektu,
- eliminacja instrukcji warunkowych,
- osobne klasy do reprezentacji każdego stanu,
- łatwe dodawanie nowego stanu,
- stany między sobą nie wiedzą o swoim istnieniu.